Reverse Engineering Preprocessor Final Design Report

Table of Contents

1. Project Description
2. User Interface Specification
3. Test Plan and Test Results
4. User Manual
5. Spring Final Presentation
6. Final Expo Poster
7. Fall Self Assessment
8. Spring Self Assessment
9. Summary of Hours
10. Summary of Expenses

Project Description

This report acts as a summary of the work completed towards the Reverse Engineering Preprocessor (REPr). REPr is a flexible and modular file identification tool for use in reverse engineering applications. The tool is intended to ease common pain points:

- Reverse engineers often encounter unknown file formats which require specialized research to decode
- Information may be hidden inside of otherwise standard file formats
- File extraction pipelines can require multiple steps, which are often performed manually

User Interface Specification

This program is run as a command-line utility, the interface is minimal to allow users to quickly execute functions.

Test Plan and Results

The results from testing indicate that the program is working as intended. Three test categories for integration with RE tools were skipped as this functionality has yet to be implemented. The unit tests were also not completed.

User Manual

See link to manual above, below are screenshots from the manual.

Usage

Jacob Greenberg edited this page 2 minutes ago · 3 revisions

The RE Preprocessor has three main usage modes:

- identify (`-I` / `--identify`): returns a list of confidence scores for each available file type
- extract (`-E` / `--extract`): identifies the file and then attempts to extract it. Supplement with the `-r/--recursive` flag for nested files.
- informative (`-info` / `--information`): provides additional details about the file, such as entropy

All function require a file to run on (`-i </path/to/file>` / `--infile </path/to/file>`)
The extract function also requires a place to put the output (`-o </path/to/file>` / `--outfile </path/to/file>`

Below are some example usages:

```
python3 main.py --identify --infile ./test_files/plain.txt.zip
python3 main.py --information --infile ./test_files/archive_plain.tar
python3 main.py --extract --infile ./test_files/plain.txt.gz --outfile /tmp/plain.txt.gz.out
```

Next up: Creating a Plugin

FAQ

- How does this project differ from other file identification utilities like Binwalk?
  This tool serves a similar niche to Binwalk as a first-line identifier. The difference is flexibility, REPr allows you to specify the file types you'd like to look for and makes it easier to add new file types through the plugin system. This tool isn't intended to replace Binwalk, but to supplement it as a quick-and-dirty way to check a file.
- Does this tool support nested files (e.g. a .tar file in a .zip file)?
  Yes! This tool recursively attempts to extract files until there's nothing left
- Can this tool detect malware?
  No. While this tool might be useful for malware reverse engineering, it isn't intended to detect malware or act as an anti-virus

Initial Self Assessment

What Worked?

- MVP established quickly
- Design documents helped to guide implementation

What needs improvement?

- Implementation often took a back seat to assignments from other courses
- More consistent communication

Spring Self Assessment

My project is an advanced file identification and processing utility. The goal is to reduce the effort required to go from a raw binary to an executable program, mainly for use in reverse engineering tasks. I chose this project because it would give me the opportunity to learn more about how Python's library system works, and how files are identified by a computer. I feel I accomplished these two learning objectives.

I would consider my project a success, but there are a few things I planned to do and was unable to. I completed the plugin system, which was a big goal, and some basic file

identification plugins. The project works, but the plugins I created were not as in depth as I was originally planning. It turns out that file identification is a hairy issue and I had not realized just how complicated definitive file identification is. My plugins provide a good jumping off point for future development at least.

Summary of Hours

Throughout the semester, I chipped away at this project dedicating a few hours of work roughly every week. The combined work well exceeds the required minimum of 45 hours of effort for the semester.

Summary of Expenses

This project did not require any infrastructure and had no expenses.